API Financeiro Alle

Integração interna com cadastros, comissões, recebimentos, despesas e indicadores.

VERSÃO 1.0.0 · BASE /api/v1 · 21/10/2026 · USO INTERNO

Comece por aqui

Configure API_TOKEN no servidor, reinicie a aplicação e envie Authorization: Bearer em cada requisição. Este manual contém as rotas, os campos e exemplos executáveis de consumo.

Desenvolvimento: `http://localhost:3000/api/v1`

Índice

1. Conexão e autenticação

2. Formato, consultas e respostas

3. Catálogo completo de rotas

4. Cadastros de apoio

5. Contas a receber

6. Contas a pagar

7. Comissões e integração financeira

8. Dashboard e sistema

9. Exemplos cURL, JavaScript e Python

10. Sequência de integração

11. Erros e arquivos entregues

Exemplos fictícios. Use os IDs retornados pela sua instalação. As importações de planilhas permanecem na interface do sistema; esta API recebe JSON.

Conexão e autenticação

Configure uma credencial interna no servidor e comece pelas consultas de descoberta e saúde.

1. Configurar o servidor

Na raiz do projeto, execute o comando abaixo: ele gera e salva API_TOKEN em .env sem exibir a credencial. Se já existir um token no arquivo, o valor é preservado.

```
npm run api:token
```

Obtenha o valor de API_TOKEN no arquivo .env do servidor e configure-o no ambiente seguro da integração. Em produção, configure a credencial como segredo/variável de ambiente do serviço. O marcador abaixo é somente ilustrativo.

```
API_TOKEN="COLE_AQUI_O_TOKEN_GERADO_PELO_COMANDO"
```

O token precisa ter pelo menos 32 caracteres e não pode conter espaços, tabulações ou quebras de linha em nenhuma posição. Reinicie a aplicação após configurar ou trocar a variável. Em desenvolvimento, use npm run dev; em produção, reconstrua/inicie conforme o processo de implantação.

2. Enviar a credencial

```
Authorization: Bearer SEU_TOKEN
Content-Type: application/json
```

URL base local: `http://localhost:3000/api/v1`. Em outro ambiente, substitua o host e mantenha `/api/v1`. Todas as rotas exigem Bearer, incluindo health e OpenAPI.

A credencial é única e permite leitura e escrita de todos os recursos. Ela não representa um usuário ou uma sessão; os perfis admin/gestor do cadastro de usuários não limitam o token. Execute as integrações em um servidor ou ferramenta interna confiável. Não coloque API_TOKEN em JavaScript servido ao navegador, aplicativos públicos ou URLs; em ambientes remotos, use HTTPS.

3. Confirmar a instalação

```
export API_BASE="http://localhost:3000/api/v1"
export ALLE_API_TOKEN="SEU_TOKEN_GERADO"
curl --fail-with-body "$API_BASE/health" \
  -H "Authorization: Bearer $ALLE_API_TOKEN"

{
  "data": {"status":"ok","servico":"financeiro-alle","versao":"1.0.0","banco":"conectado"}
}
```

GET /api/v1 informa a versão e os recursos. GET /api/v1/openapi.json devolve a especificação completa, também incluída no arquivo api-v1.openapi.json entregue com este documento.

Formato, consultas e respostas

Use JSON nos corpos de POST e PUT. A API expõe o núcleo operacional; as importações CSV/XLSX continuam disponíveis pela interface do sistema.

Assunto	Convenção
Valores monetários	Strings decimais com ponto, por exemplo `"150.50"`. Até 10 dígitos inteiros e 2 casas; sem milhar. Evite números JSON para dinheiro. Entradas e saídas exigem valor positivo; comissão permite zero. Margem pode ser negativa.
IDs de cadastro	Inteiros positivos de 1 a 2147483647; o corpo aceita número inteiro seguro ou string. Agência, conta, UC e documentos devem ser texto para preservar zeros.
IDs financeiros	Bigint sempre como string. Máximo `9223372036854775807`; inclui comissões, contas a pagar e `id_do_registro` de contas a receber. Não converta para Number no JavaScript.
Datas	Entrada `YYYY-MM-DD`; saída ISO 8601 UTC, como `2026-10-16T00:00:00.000Z`. Exiba datas civis preservando a parte YYYY-MM-DD para não deslocar o dia no fuso local.
Competência	`YYYY-MM`, por exemplo `2026-10`; os serviços financeiros e de comissão também aceitam MM/AAAA.
POST	Cria um registro; retorna 201 e {data: objeto}. IDs são gerados pelo servidor.
PUT	Substitui os campos editáveis. Envie todos os obrigatórios. Opcionais de texto/relação ausentes ou null são limpos. Flags de geração ausentes voltam a false e não aceitam null. Não é atualização parcial. PATCH não é aceito.
DELETE	Quando disponível, exclui definitivamente e retorna 204 sem corpo. Vínculos podem impedir a exclusão. Não há DELETE para comissões.
Validação	Campos desconhecidos, queries desconhecidas e parâmetros repetidos são rejeitados. Corpo máximo 64 KB. Use Content-Type: application/json.

Paginação e filtros

As coleções usam pagina=1 e limite=20 por padrão; limite máximo de 100. Filtros são combinados. As datas inicial e final são inclusivas. Consultar uma página além do total retorna data vazio; navegue pelo total de registros, sem assumir que uma página cheia implica outra página.

```
{
  "data": [
    {"id":1,"nome":"Distribuidora Exemplo"}
  ],
  "meta": {"pagina":1,"limite":20,"total":1,"paginas":1}
}
```

meta.total conta todos os resultados filtrados. Nas contas financeiras, meta.valor_total soma todos eles. Em comissões, meta.totais soma economia, empresa, parceiro e margem. Esses totais não se limitam à página atual. Cadastros são ordenados por nome/ID crescentes; contas a receber por data decrescente, datas nulas ao final; contas a pagar por vencimento/ID decrescentes.

Exemplo de filtros

```
http://localhost:3000/api/v1/comissoes?competencia=2026-
10&status_recebimento=Pendente&pagina=1&limite=50
```

Codifique os valores da query com URLSearchParams ou equivalente; por exemplo, Não se aplica contém espaços e acento. Filtros são aceitos somente no GET da coleção; detalhe e escrita não aceitam query string.

Catálogo completo de rotas

Todos os caminhos abaixo são relativos à URL base /api/v1. {id} é o identificador retornado pela criação ou listagem.

Módulo	Caminho	Métodos e finalidade
Clientes	`/clientes`	GET: listar · POST: criar
	`/clientes/{id}`	GET: consultar · PUT: atualizar · DELETE: excluir
Parceiros	`/parceiros`	GET: listar · POST: criar
	`/parceiros/{id}`	GET: consultar · PUT: atualizar · DELETE: excluir
Distribuidoras	`/distribuidoras`	GET: listar · POST: criar
	`/distribuidoras/{id}`	GET: consultar · PUT: atualizar · DELETE: excluir
Geradoras	`/geradoras`	GET: listar · POST: criar
	`/geradoras/{id}`	GET: consultar · PUT: atualizar · DELETE: excluir
Unidades Allp Fit	`/unidades`	GET: listar · POST: criar
	`/unidades/{id}`	GET: consultar · PUT: atualizar · DELETE: excluir
Contas bancárias	`/contas-bancarias`	GET: listar · POST: criar
	`/contas-bancarias/{id}`	GET: consultar · PUT: atualizar · DELETE: excluir
Métodos de pagamento	`/metodos-pagamento`	GET: listar · POST: criar
	`/metodos-pagamento/{id}`	GET: consultar · PUT: atualizar · DELETE: excluir
Tipos financeiros	`/tipos`	GET: listar · POST: criar
	`/tipos/{id}`	GET: consultar · PUT: atualizar · DELETE: excluir
Usuários	`/usuarios`	GET: listar · POST: criar
	`/usuarios/{id}`	GET: consultar · PUT: atualizar · DELETE:

Módulo	Caminho	Métodos e finalidade
		excluir
Contas a receber	`/contas-a-receber`	GET: listar · POST: criar
	`/contas-a-receber/{id}`	GET: consultar · PUT: atualizar · DELETE: excluir
Contas a pagar	`/contas-a-pagar`	GET: listar · POST: criar
	`/contas-a-pagar/{id}`	GET: consultar · PUT: atualizar · DELETE: excluir
Comissões	`/comissoes`	GET: listar · POST: criar
	`/comissoes/{id}`	GET: consultar · PUT: atualizar · sem DELETE
Comissões · lote	`/comissoes/recebimentos`	POST: registrar recebimentos em lote
Comissões · lote	`/comissoes/pagamentos`	POST: atualizar pagamentos em lote
Sistema	`/`	GET: recursos e versão
Sistema	`/health`	GET: conexão com banco
Contrato	`/openapi.json`	GET: OpenAPI 3.0.3
Indicadores	`/dashboard`	GET: indicadores financeiros

Cadastros de apoio

Cadastre as relações antes de criar clientes, comissões ou movimentos financeiros. Todos os exemplos são fictícios; substitua os IDs pelos retornados no seu ambiente.

Clientes

`GET /clientes · POST /clientes · GET /clientes/{id} · PUT /clientes/{id} · DELETE /clientes/{id}`

Clientes e unidades consumidoras. UC é única, preserva zeros iniciais e é normalizada sem espaços e em maiúsculas. CPF é opcional, validado e salvo com 11 dígitos. Clientes novos são ativos por padrão. Inativos não podem ter novas comissões; o histórico é preservado. Relações parceiro, distribuidora e unidade são devolvidas nas consultas.

Campos do JSON · POST e PUT

Campo	Tipo	Obrig.	Formato e regra
`nome`	string	Sim	Nome do cliente. Máximo 255 caracteres.
`uc`	string	Sim	UC/instalação única; envie como texto para preservar zeros. Máximo 100 caracteres.
`cpf`	string ou null	Não	CPF válido com ou sem máscara; vazio/null limpa o campo. Máximo 14 caracteres.
`parceiro_id`	inteiro ou string ou null	Não	ID do cadastro (inteiro de 1 a 2147483647).
`distribuidora_id`	inteiro ou string ou null	Não	ID do cadastro (inteiro de 1 a 2147483647).
`unidade_id`	inteiro ou string ou null	Não	ID do cadastro (inteiro de 1 a 2147483647).
`ativo`	boolean	Não	Situação atual. Na criação, omitir usa true; na edição, omitir preserva a situação.

Filtros · GET da coleção

Query string	Tipo	Uso
`pagina`	inteiro	Página a partir de 1; padrão 1.
`limite`	inteiro	De 1 a 100 registros; padrão 20.
`busca`	string	Pesquisa por nome, UC ou CPF com/sem máscara; máximo 255 caracteres.
`ativo`	string	true para ativos ou false para inativos. Omitir retorna ambos.

Query string	Tipo	Uso
`distribuidora_id`	string	ID do cadastro relacionado ou sem-vinculo para selecionar os clientes sem associação.
`unidade_id`	string	ID do cadastro relacionado ou sem-vinculo para selecionar os clientes sem associação.
`parceiro_id`	string	ID do cadastro relacionado ou sem-vinculo para selecionar os clientes sem associação.

Exemplo completo de entrada

POST `/clientes`. No PUT, envie os mesmos campos editáveis; use o ID retornado na URL.

```
{
  "nome": "Cliente Exemplo",
  "uc": "000123456",
  "cpf": null,
  "parceiro_id": null,
  "distribuidora_id": 1,
  "unidade_id": 1,
  "ativo": ""
}
```

Exemplo de resposta 201 (POST) ou 200 (GET por ID / PUT)

```
{
  "data": {
    "nome": "Cliente Exemplo",
    "uc": "000123456",
    "cpf": null,
    "parceiro_id": null,
    "distribuidora_id": 1,
    "unidade_id": 1,
    "ativo": "",
    "id": 1,
    "parceiro": null,
    "distribuidora": {"nome":"Distribuidora Exemplo","id":1},
    "unidade": {
      "nome": "Unidade Exemplo",
      "parceiro_id": 1,
      "id": 1,
      "parceiro": {
        "nome": "Parceiro Exemplo",
        "documento": null,
        "email": "parceiro@example.com",
        "telefone": "31999990000",
        "id": 1,
        "criado_em": "2026-10-16T00:00:00.000Z"
      }
    }
  }
}
```

Parceiros

`GET /parceiros · POST /parceiros · GET /parceiros/{id} · PUT /parceiros/{id} · DELETE /parceiros/{id}`

Parceiros, gestores e favorecidos. CNPJ opcional numérico ou alfanumérico, validado e único; não consulta a Receita Federal. Nomes iguais são permitidos.

Campos do JSON · POST e PUT

Campo	Tipo	Obrig.	Formato e regra
`nome`	string	Sim	Nome/razão social. Máximo 255 caracteres.
`documento`	string ou null	Não	CNPJ válido opcional, com/sem máscara. Máximo 30 caracteres.
`email`	string ou null	Não	E-mail válido opcional, normalizado em minúsculas. Máximo 255 caracteres.
`telefone`	string ou null	Não	Telefone opcional. Máximo 30 caracteres.

Filtros · GET da coleção

Query string	Tipo	Uso
`pagina`	inteiro	Página a partir de 1; padrão 1.
`limite`	inteiro	De 1 a 100 registros; padrão 20.
`busca`	string	Pesquisa parcial por nome, sem diferenciar maiúsculas/minúsculas; máximo 255 caracteres.

Exemplo completo de entrada

POST `/parceiros`. No PUT, envie os mesmos campos editáveis; use o ID retornado na URL.

```
{
  "nome": "Parceiro Exemplo",
  "documento": null,
  "email": "parceiro@example.com",
  "telefone": "31999990000"
}
```

Exemplo de resposta 201 (POST) ou 200 (GET por ID / PUT)

```
{
  "data": {
    "nome": "Parceiro Exemplo",
    "documento": null,
    "email": "parceiro@example.com",
    "telefone": "31999990000",
    "id": 1,
    "criado_em": "2026-10-16T00:00:00.000Z"
  }
}
```

Distribuidoras

`GET /distribuidoras · POST /distribuidoras · GET /distribuidoras/{id} · PUT /distribuidoras/{id} · DELETE /distribuidoras/{id}`

Distribuidoras que atendem as UCs. Nome único. Exclusão respeita vínculos com clientes e comissões.

Campos do JSON · POST e PUT

Campo	Tipo	Obrig.	Formato e regra
`nome`	string	Sim	Nome único da distribuidora. Máximo 255 caracteres.

Filtros · GET da coleção

Query string	Tipo	Uso
`pagina`	inteiro	Página a partir de 1; padrão 1.
`limite`	inteiro	De 1 a 100 registros; padrão 20.
`busca`	string	Pesquisa parcial por nome, sem diferenciar maiúsculas/minúsculas; máximo 255 caracteres.

Exemplo completo de entrada

POST `/distribuidoras`. No PUT, envie os mesmos campos editáveis; use o ID retornado na URL.

```
{"nome":"Distribuidora Exemplo"}
```

Exemplo de resposta 201 (POST) ou 200 (GET por ID / PUT)

```
{"data":{"nome":"Distribuidora Exemplo","id":1}}
```

Geradoras

`GET /geradoras · POST /geradoras · GET /geradoras/{id} · PUT /geradoras/{id} · DELETE /geradoras/{id}`

Geradoras pagadoras de comissões à Alle. Nome único. Exclusão respeita os vínculos com comissões.

Campos do JSON · POST e PUT

Campo	Tipo	Obrig.	Formato e regra
`nome`	string	Sim	Nome único da geradora. Máximo 255 caracteres.

Filtros · GET da coleção

Query string	Tipo	Uso
`pagina`	inteiro	Página a partir de 1; padrão 1.

Query string	Tipo	Uso
`limite`	inteiro	De 1 a 100 registros; padrão 20.
`busca`	string	Pesquisa parcial por nome, sem diferenciar maiúsculas/minúsculas; máximo 255 caracteres.

Exemplo completo de entrada

POST `/geradoras`. No PUT, envie os mesmos campos editáveis; use o ID retornado na URL.

```
{"nome":"Geradora Exemplo"}
```

Exemplo de resposta 201 (POST) ou 200 (GET por ID / PUT)

```
{"data":{"nome":"Geradora Exemplo","id":1}}
```

Unidades Allp Fit

`GET /unidades · POST /unidades · GET /unidades/{id} · PUT /unidades/{id} · DELETE /unidades/{id}`

Unidades de origem da indicação. O parceiro associado representa seu gestor; relação parceiro incluída nas consultas.

Campos do JSON · POST e PUT

Campo	Tipo	Obrig.	Formato e regra
`nome`	string	Sim	Nome único da unidade. Máximo 255 caracteres.
`parceiro_id`	inteiro ou string ou null	Não	ID do cadastro (inteiro de 1 a 2147483647).

Filtros · GET da coleção

Query string	Tipo	Uso
`pagina`	inteiro	Página a partir de 1; padrão 1.
`limite`	inteiro	De 1 a 100 registros; padrão 20.
`busca`	string	Pesquisa parcial por nome, sem diferenciar maiúsculas/minúsculas; máximo 255 caracteres.

Exemplo completo de entrada

POST `/unidades`. No PUT, envie os mesmos campos editáveis; use o ID retornado na URL.

```
{"nome":"Unidade Exemplo","parceiro_id":1}
```

Exemplo de resposta 201 (POST) ou 200 (GET por ID / PUT)


```
{
  "data": {
    "nome": "Unidade Exemplo",
    "parceiro_id": 1,
    "id": 1,
    "parceiro": {
      "nome": "Parceiro Exemplo",
      "documento": null,
      "email": "parceiro@example.com",
      "telefone": "31999990000",
      "id": 1,
      "criado_em": "2026-10-16T00:00:00.000Z"
    }
  }
}
```


Contas bancárias

`GET /contas-bancarias · POST /contas-bancarias · GET /contas-bancarias/{id} · PUT /contas-bancarias/{id} · DELETE /contas-bancarias/{id}`

Contas de recebimento. Banco/agência/número formam uma chave única. Banco, agência e número são normalizados em maiúsculas; agência e número preservam zeros e dígitos alfanuméricos quando enviados como texto.

Campos do JSON · POST e PUT

Campo	Tipo	Obrig.	Formato e regra
`nome`	string	Sim	Identificação da conta. Máximo 100 caracteres.
`banco`	string	Sim	Nome/código do banco. Máximo 100 caracteres.
`agencia`	string	Sim	Agência, como texto. Máximo 20 caracteres.
`numero`	string	Sim	Número com dígito, como texto. Máximo 30 caracteres.
`titular`	string	Sim	Titular/beneficiário. Máximo 255 caracteres.

Filtros · GET da coleção

Query string	Tipo	Uso
`pagina`	inteiro	Página a partir de 1; padrão 1.
`limite`	inteiro	De 1 a 100 registros; padrão 20.
`busca`	string	Pesquisa parcial por nome, sem diferenciar maiúsculas/minúsculas; máximo 255 caracteres.

Exemplo completo de entrada

POST `/contas-bancarias`. No PUT, envie os mesmos campos editáveis; use o ID retornado na URL.

```
{
  "nome": "Conta Operacional",
  "banco": "001",
  "agencia": "0001",
  "numero": "12345-6",
  "titular": "Empresa Exemplo"
}
```

Exemplo de resposta 201 (POST) ou 200 (GET por ID / PUT)

```
{
  "data": {
    "nome": "Conta Operacional",
    "banco": "001",
    "agencia": "0001",
    "numero": "12345-6",
    "titular": "Empresa Exemplo",
    "id": 1
  }
}
```

Métodos de pagamento

`GET /metodos-pagamento · POST /metodos-pagamento · GET /metodos-pagamento/{id} · PUT /metodos-pagamento/{id} · DELETE /metodos-pagamento/{id}`

Métodos de recebimento com nome único normalizado em maiúsculas.

Campos do JSON · POST e PUT

Campo	Tipo	Obrig.	Formato e regra
`nome`	string	Sim	Nome do método. Máximo 100 caracteres.

Filtros · GET da coleção

Query string	Tipo	Uso
`pagina`	inteiro	Página a partir de 1; padrão 1.
`limite`	inteiro	De 1 a 100 registros; padrão 20.
`busca`	string	Pesquisa parcial por nome, sem diferenciar maiúsculas/minúsculas; máximo 255 caracteres.

Exemplo completo de entrada

POST `/metodos-pagamento`. No PUT, envie os mesmos campos editáveis; use o ID retornado na URL.

```
{"nome":"PIX"}
```

Exemplo de resposta 201 (POST) ou 200 (GET por ID / PUT)

```
{"data":{"nome":"PIX","id":1}}
```

Tipos financeiros

`GET /tipos · POST /tipos · GET /tipos/{id} · PUT /tipos/{id} · DELETE /tipos/{id}`

Classificação dos lançamentos financeiros. Nome único e preservado conforme o validador atual.

Campos do JSON · POST e PUT

Campo	Tipo	Obrig.	Formato e regra
`nome`	string	Sim	Nome do tipo financeiro. Máximo 100 caracteres.

Filtros · GET da coleção

Query string	Tipo	Uso
`pagina`	inteiro	Página a partir de 1; padrão 1.
`limite`	inteiro	De 1 a 100 registros; padrão 20.
`busca`	string	Pesquisa parcial por nome, sem diferenciar maiúsculas/minúsculas; máximo 255 caracteres.

Exemplo completo de entrada

POST `/tipos`. No PUT, envie os mesmos campos editáveis; use o ID retornado na URL.

```
{"nome":"Serviços"}
```

Exemplo de resposta 201 (POST) ou 200 (GET por ID / PUT)

```
{"data":{"nome":"Serviços","id":1}}
```

Usuários

`GET /usuarios · POST /usuarios · GET /usuarios/{id} · PUT /usuarios/{id} · DELETE /usuarios/{id}`

Cadastro de usuários, sem login ou sessão. O perfil cadastral não altera as permissões do token da API. E-mail único e normalizado em minúsculas.

Campos do JSON · POST e PUT

Campo	Tipo	Obrig.	Formato e regra
`nome`	string	Sim	Nome do usuário. Máximo 255 caracteres.
`email`	string · e-mail	Sim	E-mail válido e único. Máximo 255 caracteres.
`role`	string	Sim	Perfil cadastral. Valores: admin | gestor.

Filtros · GET da coleção

Query string	Tipo	Uso
`pagina`	inteiro	Página a partir de 1; padrão 1.
`limite`	inteiro	De 1 a 100 registros; padrão 20.
`busca`	string	Pesquisa parcial por nome, sem diferenciar maiúsculas/minúsculas; máximo 255 caracteres.

Exemplo completo de entrada

POST `/usuarios`. No PUT, envie os mesmos campos editáveis; use o ID retornado na URL.

```
{"nome":"Gestor Exemplo","email":"gestor@example.com","role":"gestor"}
```

Exemplo de resposta 201 (POST) ou 200 (GET por ID / PUT)

```
{
  "data": {
    "nome": "Gestor Exemplo",
    "email": "gestor@example.com",
    "role": "gestor",
    "id": 1,
    "criado_em": "2026-10-16T00:00:00.000Z"
  }
}
```

Contas a receber

Recebimentos independentes e consulta das entradas financeiras geradas por comissões.

Contas a receber

`GET /contas-a-receber · POST /contas-a-receber · GET /contas-a-receber/{id} · PUT /contas-a-receber/{id} · DELETE /contas-a-receber/{id}`

Recebimentos. O ID da URL é id_do_registro, nunca id_do_pagamento. O servidor preenche beneficiario pelo titular da conta e metodo_do_pagamento pelo método selecionado. Consultas incluem tipo, parceiro, conta_bancaria, metodo e comissao:{id}| null. Movimentos ligados a comissão devem ser alterados pelo recurso comissoes.

O identificador usado em `/{id}` é **id_do_registro**. `id_do_pagamento` é apenas informativo. `beneficiario`, `pagador`, `descricao` e `metodo_do_pagamento` não são campos graváveis nesta rota. Recebido pode ter data nula conforme o serviço atual; sem data, fica fora dos indicadores por período.

Campos do JSON · POST e PUT

Campo	Tipo	Obrig.	Formato e regra
`parceiro_id`	inteiro ou string	Sim	ID do cadastro (inteiro de 1 a 2147483647).
`tipo_id`	inteiro ou string	Sim	ID do cadastro (inteiro de 1 a 2147483647).
`valor_total`	string	Sim	Valor decimal em reais, sem separador de milhar. Use string e ponto decimal; até 10 dígitos inteiros e 2 casas. Entradas/saídas exigem valor maior que zero; comissões permitem zero.
`data_do_pagamento`	string · data ou null	Não	Data civil no formato YYYY-MM-DD.
`competencia_financeira`	string	Sim	Competência YYYY-MM. Também é aceito MM/AAAA nos serviços financeiros e de comissão.
`conta_bancaria_id`	inteiro ou string	Sim	ID do cadastro (inteiro de 1 a 2147483647).
`metodo_id`	inteiro ou string	Sim	ID do cadastro (inteiro de 1 a 2147483647).
`status_do_pagamento`	string	Sim	Status do recebimento. Valores: Recebido | Pendente.
`numero_da_nf`	string ou null	Não	Número da nota fiscal, opcional. Máximo 100 caracteres.

Filtros · GET da coleção

Query string	Tipo	Uso
`pagina`	inteiro	Página a partir de 1; padrão 1.

Query string	Tipo	Uso
`limite`	inteiro	De 1 a 100 registros; padrão 20.
`inicio`	string · data	Data inicial inclusiva YYYY-MM-DD.
`fim`	string · data	Data final inclusiva YYYY-MM-DD.
`parceiro`	integer	ID do parceiro.
`conta`	integer	ID da conta bancária.
`metodo`	integer	ID do método.
`tipo`	integer	ID do tipo financeiro.
`status`	string	Recebido ou Pendente.

A listagem inclui `meta.valor_total`: soma de todos os registros filtrados, independentemente da página.

Exemplo completo de entrada

POST `/contas-a-receber`. No PUT, envie os mesmos campos editáveis; use o ID retornado na URL.

```
{
  "parceiro_id": 1,
  "tipo_id": 1,
  "valor_total": "150.50",
  "data_do_pagamento": "2026-10-16",
  "competencia_financeira": "2026-10",
  "conta_bancaria_id": 1,
  "metodo_id": 1,
  "status_do_pagamento": "Recebido",
  "numero_da_nf": "NF-123"
}
```

Exemplo de resposta 201 (POST) ou 200 (GET por ID / PUT)


```
{
  "data": {
    "parceiro_id": 1,
    "tipo_id": 1,
    "valor_total": "150.50",
    "data_do_pagamento": "2026-10-16T00:00:00.000Z",
    "competencia_financeira": "2026-10",
    "conta_bancaria_id": 1,
    "metodo_id": 1,
    "status_do_pagamento": "Recebido",
    "numero_da_nf": "NF-123",
    "id_do_registro": "1",
    "id_do_pagamento": "1",
    "beneficiario": "Empresa Exemplo",
    "pagador": null,
    "metodo_do_pagamento": "PIX",
    "descricao": null,
    "tipo": {"nome":"Serviços","id":1},
    "parceiro": {
      "nome": "Parceiro Exemplo",
      "documento": null,
      "email": "parceiro@example.com",
      "telefone": "31999990000",
      "id": 1,
      "criado_em": "2026-10-16T00:00:00.000Z"
```

```
    },
    "conta_bancaria": {
      "nome": "Conta Operacional",
      "banco": "001",
      "agencia": "0001",
      "numero": "12345-6",
      "titular": "Empresa Exemplo",
      "id": 1
    },
    "metodo": {"nome":"PIX","id":1},
    "comissao": null
  }
}
```

Contas a pagar

Despesas independentes e consulta das saídas financeiras geradas por comissões.

Contas a pagar

`GET /contas-a-pagar · POST /contas-a-pagar · GET /contas-a-pagar/{id} · PUT /contas-a-pagar/{id} · DELETE /contas-a-pagar/{id}`

Despesas com favorecido. Todas as datas de escrita usam YYYY-MM-DD. Consultas incluem tipo, favorecido e comissao:{id}|null. Movimentos ligados a comissão devem ser alterados pelo recurso comissoes.

`centro_custo` é campo legado de leitura; não é aceito na escrita desta API. Filtros de datas usam `vencimento`.

Campos do JSON · POST e PUT

Campo	Tipo	Obrig.	Formato e regra
`favorecido_id`	inteiro ou string	Sim	ID do cadastro (inteiro de 1 a 2147483647).
`descricao`	string	Sim	Descrição da despesa. Máximo 500 caracteres.
`tipo_id`	inteiro ou string	Sim	ID do cadastro (inteiro de 1 a 2147483647).
`lancamento`	string · data	Sim	Data civil no formato YYYY-MM-DD.
`competencia`	string	Sim	Competência YYYY-MM. Também é aceito MM/AAAA nos serviços financeiros e de comissão.
`vencimento`	string · data	Sim	Data civil no formato YYYY-MM-DD.
`valor`	string	Sim	Valor decimal em reais, sem separador de milhar. Use string e ponto decimal; até 10 dígitos inteiros e 2 casas. Entradas/saídas exigem valor maior que zero; comissões permitem zero.
`status`	string	Sim	Status da despesa. Valores: Pago | Pendente.

Filtros · GET da coleção

Query string	Tipo	Uso
`pagina`	inteiro	Página a partir de 1; padrão 1.
`limite`	inteiro	De 1 a 100 registros; padrão 20.
`inicio`	string · data	Data inicial inclusiva YYYY-MM-DD.
`fim`	string · data	Data final inclusiva YYYY-MM-DD.

Query string	Tipo	Uso
`favorecido`	integer	ID do parceiro favorecido.
`descricao`	string	Trecho da descrição, até 500 caracteres; ignora maiúsculas/minúsculas.
`tipo`	integer	ID do tipo financeiro.
`competencia`	string	Competência YYYY-MM ou MM/AAAA.
`status`	string	Pago ou Pendente.

A listagem inclui `meta.valor_total`: soma de todos os registros filtrados, independentemente da página.

Exemplo completo de entrada

POST `/contas-a-pagar`. No PUT, envie os mesmos campos editáveis; use o ID retornado na URL.

```
{
  "favorecido_id": 1,
  "descricao": "Serviço contratado",
  "tipo_id": 1,
  "lancamento": "2026-10-16",
  "competencia": "2026-10",
  "vencimento": "2026-10-20",
  "valor": "100.00",
  "status": "Pendente"
}
```

Exemplo de resposta 201 (POST) ou 200 (GET por ID / PUT)

```
{
  "data": {
    "favorecido_id": 1,
    "descricao": "Serviço contratado",
    "tipo_id": 1,
    "lancamento": "2026-10-16T00:00:00.000Z",
    "competencia": "2026-10",
    "vencimento": "2026-10-20T00:00:00.000Z",
    "valor": "100.00",
    "status": "Pendente",
    "id": "1",
    "centro_custo": null,
    "tipo": {"nome":"Serviços","id":1},
    "favorecido": {
      "nome": "Parceiro Exemplo",
      "documento": null,
      "email": "parceiro@example.com",
      "telefone": "31999990000",
      "id": 1,
      "criado_em": "2026-10-16T00:00:00.000Z"
    },
    "comissao": null
  }
}
```

Comissões e integração financeira

A API não oferece exclusão de comissões. Use os campos de status e os vínculos financeiros para manter a consistência dos lançamentos.

Comissões

`GET /comissoes · POST /comissoes · GET /comissoes/{id} · PUT /comissoes/{id}`

Comissão por cliente/UC, geradora e competência, sem duplicidade. Margem calculada no servidor (empresa menos parceiro). Relações cliente/unidade, geradora, distribuidora e parceiro incluídas. Gravação e movimentos financeiros ocorrem na mesma transação. A API não oferece exclusão de comissão.

Somente leitura: `id`, `margem`, `entrada_id` e `saida_id`. Não envie esses campos. As flags são booleanos JSON; `"on"` não é aceito.

Campos do JSON · POST e PUT

Campo	Tipo	Obrig.	Formato e regra
`cliente_id`	inteiro ou string	Sim	ID do cadastro (inteiro de 1 a 2147483647).
`geradora_id`	inteiro ou string	Sim	ID do cadastro (inteiro de 1 a 2147483647).
`distribuidora_id`	inteiro ou string	Sim	ID do cadastro (inteiro de 1 a 2147483647).
`parceiro_id`	inteiro ou string ou null	Não	ID do cadastro (inteiro de 1 a 2147483647).
`competencia`	string	Sim	Competência YYYY-MM. Também é aceito MM/AAAA nos serviços financeiros e de comissão.
`valor_economia`	string	Sim	Valor decimal em reais, sem separador de milhar. Use string e ponto decimal; até 10 dígitos inteiros e 2 casas. Entradas/saídas exigem valor maior que zero; comissões permitem zero.
`valor_comissao_empresa`	string	Sim	Valor decimal em reais, sem separador de milhar. Use string e ponto decimal; até 10 dígitos inteiros e 2 casas. Entradas/saídas exigem valor maior que zero; comissões permitem zero.
`valor_comissao_parceiro`	string	Não	Valor calculado pelo servidor: R$ 2,00, mesmo sem unidade/parceiro vinculado. O campo é opcional e valores enviados são ignorados. Edições de comissões já pagas preservam o valor histórico.
`status_recebimento`	string	Sim	Recebido exige data_recebimento. Valores: Pendente | Recebido.

Campo	Tipo	Obrig.	Formato e regra
`status_pagamento`	string	Sim	Com parceiro: Pendente ou Pago. Sem parceiro, o valor de R$ 2,00 permanece Pendente; vincule o beneficiário antes de pagar. Não se aplica é aceito para compatibilidade e convertido em Pendente na gravação. Valores: Pendente | Pago | Não se aplica.
`data_recebimento`	string · data ou null	Não	Data civil no formato YYYY-MM-DD.
`data_pagamento`	string · data ou null	Não	Data civil no formato YYYY-MM-DD.
`gerar_entrada`	boolean	Não	true gera recebimento; exige status Recebido, conta_bancaria_id, metodo_id e valor positivo. Padrão: false.
`gerar_saida`	boolean	Não	true gera despesa; exige parceiro, status Pago, data_pagamento e valor positivo. Padrão: false.
`conta_bancaria_id`	inteiro ou string ou null	Não	ID do cadastro (inteiro de 1 a 2147483647).
`metodo_id`	inteiro ou string ou null	Não	ID do cadastro (inteiro de 1 a 2147483647).
`entrada_existente_id`	string ou null	Não	ID bigint positivo, máximo 9223372036854775807. Use string para preservar precisão.
`saida_existente_id`	string ou null	Não	ID bigint positivo, máximo 9223372036854775807. Use string para preservar precisão.

Filtros · GET da coleção

Query string	Tipo	Uso
`pagina`	inteiro	Página a partir de 1; padrão 1.
`limite`	inteiro	De 1 a 100 registros; padrão 20.
`competencia`	string	Competência YYYY-MM ou MM/AAAA.
`geradora_id`	integer	ID do cadastro relacionado.
`distribuidora_id`	integer	ID do cadastro relacionado.
`unidade_id`	integer	ID do cadastro relacionado.
`parceiro_id`	integer	ID do cadastro relacionado.
`status_recebimento`	string	Pendente ou Recebido.
`status_pagamento`	string	Pendente, Pago ou Não se aplica.

Query string	Tipo	Uso
`busca`	string	Nome ou UC do cliente; máximo 255 caracteres.

A listagem inclui `meta.totais` com `valor_economia`, `valor_comissao_empresa`, `valor_comissao_parceiro` e `margem`, somados sobre todos os registros filtrados. Ordenação: competência e ID decrescentes.

Exemplo completo de entrada

POST `/comissoes`. No PUT, envie os mesmos campos editáveis; use o ID retornado na URL.

```
{
  "cliente_id": 1,
  "geradora_id": 1,
  "distribuidora_id": 1,
  "parceiro_id": 1,
  "competencia": "2026-10",
  "valor_economia": "300.00",
  "valor_comissao_empresa": "150.00",
  "valor_comissao_parceiro": "2.00",
  "status_recebimento": "Pendente",
  "status_pagamento": "Pendente",
  "data_recebimento": "2026-10-16",
  "data_pagamento": "2026-10-16",
  "gerar_entrada": false,
  "gerar_saida": false,
  "conta_bancaria_id": null,
  "metodo_id": null,
  "entrada_existente_id": null,
  "saida_existente_id": null
}
```

Exemplo de resposta 201 (POST) ou 200 (GET por ID / PUT)

```
{
  "data": {
    "cliente_id": 1,
    "geradora_id": 1,
    "distribuidora_id": 1,
    "parceiro_id": 1,
    "competencia": "2026-10",
    "valor_economia": "300.00",
    "valor_comissao_empresa": "150.00",
    "valor_comissao_parceiro": "2.00",
    "status_recebimento": "Pendente",
    "status_pagamento": "Pendente",
    "data_recebimento": "2026-10-16T00:00:00.000Z",
    "data_pagamento": "2026-10-16T00:00:00.000Z",
    "id": "1",
    "margem": "148.00",
    "entrada_id": null,
    "saida_id": null,
    "cliente": {
      "nome": "Cliente Exemplo",
      "uc": "000123456",
      "cpf": null,
      "parceiro_id": null,
      "distribuidora_id": 1,
      "unidade_id": 1,
      "ativo": "",
      "id": 1,
      "unidade": {"nome":"Unidade Exemplo","parceiro_id":1,"id":1}
    },
    "geradora": {"nome":"Geradora Exemplo","id":1},
    "distribuidora": {"nome":"Distribuidora Exemplo","id":1},
    "parceiro": {
      "nome": "Parceiro Exemplo",
      "documento": null,
      "email": "parceiro@example.com",
```

```
      "telefone": "31999990000",
      "id": 1,
      "criado_em": "2026-10-16T00:00:00.000Z"
    }
  }
}
```


Recebimento e pagamento independentes

A geradora paga a comissão à Alle; o parceiro recebe sua comissão. É possível pagar o parceiro antes de receber da geradora. Recebido exige data_recebimento; Pago exige data_pagamento. A comissão do parceiro é calculada pelo servidor em R$ 2,00, mesmo sem unidade ou parceiro vinculado. Sem beneficiário, o valor permanece Pendente e o pagamento exige o vínculo. Valores de comissões já pagas são preservados em edições. Na migração, comissões antigas ainda não pagas foram recalculadas para R$ 2,00, com ajuste da margem.

A combinação cliente/UC, geradora e competência deve ser única. A distribuidora precisa corresponder à cadastrada para o cliente; clientes sem distribuidora são associados à informada. Uma comissão antiga sem geradora bloqueia outra da mesma UC/competência até sua correção. Margem é calculada no servidor e não aceita edição direta.

Gerar movimentos financeiros

gerar_entrada=true exige status_recebimento=Recebido, data, conta_bancaria_id, metodo_id e valor_comissao_empresa positivo. gerar_saida=true exige parceiro, status_pagamento=Pago, data e valor_comissao_parceiro positivo. O servidor classifica os movimentos como Comissão e mantém a gravação vinculada à comissão na mesma transação.

Depois do vínculo, edite pelo recurso comissoes: os movimentos são sincronizados com valores, datas, competência e status. Uma entrada vinculada exige manter a comissão como Recebido. Uma saída vinculada exige manter parceiro e status Pago. Omitir flags em um PUT não remove vínculos existentes. As rotas financeiras recusam PUT/DELETE de movimentos vinculados com 409.

Vincular entradas ou saídas existentes

Campo	Condições obrigatórias
`entrada_existente_id`	A entrada deve estar Recebido e ter o mesmo valor da empresa, competência e data de recebimento. Se pagador estiver preenchido, deve identificar a geradora. Não pode estar ligada a outra comissão.
`saida_existente_id`	A saída deve estar Pago e ter o mesmo parceiro/favorecido, valor do parceiro, competência e data de pagamento. Não pode estar ligada a outra comissão.

Informe o ID existente como string. Não combine vincular e gerar no mesmo lado; também não substitua um vínculo já existente por outro. Os campos entrada_id e saida_id aparecem apenas nas respostas.

Operações em lote

Ambas exigem de 1 a 100 IDs de comissão, enviados como strings, sem repetições. Todo o lote é processado na mesma transação; um ID inexistente ou uma regra impeditiva cancela as alterações. Marcar status em lote sincroniza movimentos já vinculados e não cria novos lançamentos.

POST /comissoes/recebimentos · Registrar recebimentos em lote

Marca todas as comissões como Recebido e sincroniza data e status de entradas já vinculadas. Não cria movimentos financeiros. O lote é atômico.

Campo	Tipo	Obrig.	Formato e regra
`ids`	array de string	Sim	De 1 a 100 itens, sem repetições.
`data_recebimento`	string · data	Sim	Data civil no formato YYYY-MM-DD.

```
{"ids":["1","2"],"data_recebimento":"2026-10-16"}

{"data":{"atualizadas":2}}
```

POST /comissoes/pagamentos · Atualizar pagamentos em lote

Pago exige data_pagamento e ignora comissões sem parceiro; atualizadas pode ser menor que o lote. Pendente limpa a data e recalcula a comissão em R$ 2,00 e a margem, inclusive sem parceiro. Uma saída já vinculada impede retorno a Pendente. Atualiza datas de saídas vinculadas e não cria movimentos. O lote é atômico.

Campo	Tipo	Obrig.	Formato e regra
`ids`	array de string	Sim	De 1 a 100 itens, sem repetições.
`status_pagamento`	string	Sim	Novo status do pagamento. Valores: Pago | Pendente.
`data_pagamento`	string · data ou null	Não	Data civil no formato YYYY-MM-DD.

```
{"ids":["1","2"],"status_pagamento":"Pago","data_pagamento":"2026-10-16"}

{"data":{"atualizadas":2}}
```

Dashboard e rotas do sistema

Todas as consultas desta seção usam o mesmo token Bearer da API.

GET /dashboard

Retorna entradas Recebido por data_do_pagamento e saídas Pago por vencimento, no intervalo inclusivo. Pendências e indicadores de comissões são separados. Não some as comissões novamente ao saldo financeiro: elas podem já estar representadas por entradas/saídas vinculadas. Para o cálculo de hoje, o fuso padrão é America/Sao_Paulo.

Query string	Tipo	Uso
`periodo`	string	hoje, ontem, 7dias, mes, ano ou personalizado. Padrão: mes (intervalo móvel de um mês).
`dia`	string	Um dia específico YYYY-MM-DD, sem periodo explícito.
`inicio`	string · data	Data inicial inclusiva YYYY-MM-DD. Informe ambas com periodo=personalizado (ou sem periodo).
`fim`	string · data	Data final inclusiva YYYY-MM-DD. Informe ambas com periodo=personalizado (ou sem periodo).

```
http://localhost:3000/api/v1/dashboard?periodo=personalizado&inicio=2026-10-01&fim=2026-10-31

{
  "data": {
    "periodo": "personalizado",
    "inicio": "2026-10-01",
    "fim": "2026-10-31",
    "entradas": "150.00",
    "saidas": "50.00",
    "saldo": "100.00",
    "a_receber": "20.00",
    "a_pagar": "10.00",
    "comissoes_recebidas": "150.00",
    "comissoes_pagas": "50.00",
    "quantidade_entradas": 1,
    "quantidade_saidas": 1
  }
}
```

a_receber e a_pagar somam os lançamentos Pendente com data dentro do intervalo selecionado, inclusive datas futuras quando o período as abrange. Não representam toda a carteira fora desse intervalo. mes e ano são períodos móveis, conforme o dashboard do sistema.

GET /

```
{
  "data": {
    "nome": "Financeiro Alle API",
    "versao": "1.0.0",
    "base": "/api/v1",
```

```
    "openapi": "/api/v1/openapi.json",
    "recursos": [
      "clientes",
      "parceiros",
      "distribuidoras",
      "geradoras",
      "unidades",
      "contas-bancarias",
      "metodos-pagamento",
      "tipos",
      "usuarios",
      "contas-a-receber",
      "contas-a-pagar",
      "comissoes",
      "dashboard",
      "health"
    ]
  }
}
```


GET /health

Verifica a API e uma consulta simples de conexão ao banco. Em caso de indisponibilidade, retorna erro 503 ou erro interno conforme a falha.


```
{
  "data": {"status":"ok","servico":"financeiro-alle","versao":"1.0.0","banco":"conectado"}
}
```


GET /openapi.json

Devolve o documento OpenAPI 3.0.3 sem envelope data. Importe docs/api-v1.openapi.json em uma ferramenta compatível ou baixe a versão servida com o cabeçalho de autenticação. Não é necessário consultar o banco para obter o contrato.

Exemplos para consumir a API

Os exemplos usam variáveis de ambiente da integração. ALLE_API_TOKEN deve conter o mesmo valor configurado em API_TOKEN no servidor do sistema.

cURL

```
# Configure no terminal da integração, fora de código público.
export API_BASE="http://localhost:3000/api/v1"
export ALLE_API_TOKEN="SEU_TOKEN_GERADO"

# Listar, com paginação e filtro.
curl --fail-with-body --get "$API_BASE/clientes" \
  -H "Authorization: Bearer $ALLE_API_TOKEN" \
  --data-urlencode "busca=Cliente Exemplo" \
  --data-urlencode "pagina=1" --data-urlencode "limite=20"

# Criar uma distribuidora.
curl --fail-with-body "$API_BASE/distribuidoras" \
  -H "Authorization: Bearer $ALLE_API_TOKEN" \
  -H "Content-Type: application/json" \
  --data '{"nome":"Distribuidora Exemplo"}'

# Atualizar: substitua 1 pelo ID retornado.
curl --fail-with-body -X PUT "$API_BASE/distribuidoras/1" \
  -H "Authorization: Bearer $ALLE_API_TOKEN" \
  -H "Content-Type: application/json" \
  --data '{"nome":"Distribuidora Exemplo Atualizada"}'

# Excluir só depois de remover dependências; 204 tem corpo vazio.
curl --fail-with-body -i -X DELETE "$API_BASE/distribuidoras/1" \
  -H "Authorization: Bearer $ALLE_API_TOKEN"
```

JavaScript · fetch no servidor

```
// Execute em Node.js (fetch nativo), no servidor da integração.
const base = process.env.ALLE_API_BASE ?? "http://localhost:3000/api/v1";
const token = process.env.ALLE_API_TOKEN;
if (!token) throw new Error("Configure ALLE_API_TOKEN");

async function api(path, method = "GET", body) {
  const response = await fetch(base + path, {
    method,
    headers: {
      Authorization: "Bearer " + token,
      ...(body === undefined ? {} : { "Content-Type": "application/json" })
    },
    ...(body === undefined ? {} : { body: JSON.stringify(body) })
  });
  if (response.status === 204) return null;
  const result = await response.json();
  if (!response.ok) {
    throw new Error(response.status + " " + result.error.code + ": " +
      result.error.message);
  }
  return result;
}

const query = new URLSearchParams({
  competencia: "2026-10", pagina: "1", limite: "50"
});
```

```
const page = await api("/comissoes?" + query);
console.log(page.meta.total, page.meta.totais);

const created = await api("/distribuidoras", "POST", {
  nome: "Distribuidora Integração Exemplo"
});
console.log("ID retornado:", created.data.id);
// Conserve IDs bigint como strings e dinheiro como string decimal.
// Para paginação, repita o GET até cobrir meta.total.
```

Python · urllib, sem bibliotecas externas

```
# Python 3: somente a biblioteca padrão; execute no servidor.
import json
import os
from urllib.request import Request, urlopen
from urllib.parse import urlencode
from urllib.error import HTTPError

BASE = os.environ.get("ALLE_API_BASE", "http://localhost:3000/api/v1")
TOKEN = os.environ["ALLE_API_TOKEN"]

def api(path, method="GET", body=None):
    headers = {"Authorization": "Bearer " + TOKEN}
    data = None
    if body is not None:
        headers["Content-Type"] = "application/json"
        data = json.dumps(body).encode("utf-8")
    request = Request(BASE + path, data=data, headers=headers, method=method)
    try:
        with urlopen(request, timeout=30) as response:
            if response.status == 204:
                return None
            return json.load(response)
    except HTTPError as error:
        detail = json.loads(error.read().decode("utf-8"))
        raise RuntimeError(
            str(error.code) + " " + detail["error"]["code"] + ": " +
            detail["error"]["message"]
        ) from error

query = urlencode({"pagina": 1, "limite": 20, "busca": "Cliente Exemplo"})
page = api("/clientes?" + query)
print(page["meta"]["total"])
created = api("/distribuidoras", "POST", {"nome": "Distribuidora Python"})
print(created["data"]["id"])
# Use strings para valores monetários e preserve IDs recebidos.
```

Sequência: cadastro → comissão → financeiro

O fluxo abaixo usa os IDs retornados em cada etapa. Não presuma que um ID do exemplo exista no seu banco.

Ordem: parceiro, distribuidora, geradora, unidade, conta/método, cliente e comissão. O tipo Comissão é selecionado/criado pelo serviço quando um movimento financeiro é gerado; movimentos independentes usam um tipo cadastrado por /tipos.

```
// Reutilize a função api(path, method, body) do exemplo JavaScript.
// Todos os nomes são fictícios. Em nova execução, consulte/reutilize
// cadastros existentes para evitar duplicidade de nomes e UC.
const parceiro = (await api("/parceiros", "POST", {
  nome: "Gestor Exemplo Fluxo", email: "gestor.fluxo@example.com"
})).data;
const distribuidora = (await api("/distribuidoras", "POST", {
  nome: "Distribuidora Exemplo Fluxo"
})).data;
const geradora = (await api("/geradoras", "POST", {
  nome: "Geradora Exemplo Fluxo"
})).data;
const unidade = (await api("/unidades", "POST", {
  nome: "Unidade Exemplo Fluxo", parceiro_id: parceiro.id
})).data;
const conta = (await api("/contas-bancarias", "POST", {
  nome: "Conta Fluxo", banco: "001", agencia: "0001",
  numero: "12345-6", titular: "Empresa Exemplo Fluxo"
})).data;
// PIX pode já existir: em integrações recorrentes, busque e reutilize.
const metodo = (await api("/metodos-pagamento", "POST", {
  nome: "PIX FLUXO EXEMPLO"
})).data;
const cliente = (await api("/clientes", "POST", {
  nome: "Cliente Exemplo Fluxo", uc: "000123456FLUXO",
  distribuidora_id: distribuidora.id, unidade_id: unidade.id,
  parceiro_id: parceiro.id
})).data;

const dados = {
  cliente_id: cliente.id, geradora_id: geradora.id,
  distribuidora_id: distribuidora.id, parceiro_id: parceiro.id,
  competencia: "2026-10", valor_economia: "300.00",
  valor_comissao_empresa: "150.00", valor_comissao_parceiro: "2.00",
  status_recebimento: "Pendente", status_pagamento: "Pendente",
  gerar_entrada: false, gerar_saida: false
};
const comissao = (await api("/comissoes", "POST", dados)).data;

// Registrar a quitação e gerar os dois movimentos na mesma transação.
const quitada = (await api("/comissoes/" + comissao.id, "PUT", {
  ...dados, status_recebimento: "Recebido", status_pagamento: "Pago",
  data_recebimento: "2026-10-16", data_pagamento: "2026-10-16",
  gerar_entrada: true, gerar_saida: true,
  conta_bancaria_id: conta.id, metodo_id: metodo.id
})).data;

// O ID da entrada é id_do_registro. Ambos os vínculos são strings.
const entrada = await api("/contas-a-receber/" + quitada.entrada_id);
const saida = await api("/contas-a-pagar/" + quitada.saida_id);
console.log(quitada.margem, entrada.data, saida.data);
// Esperado: margem "148.00"; recebimento "150.00"; despesa "2.00".
// Edite essa comissão por /comissoes/{id} para preservar a sincronização.
```

Alternativa: crie/consulte um movimento financeiro existente e envie entrada_existente_id ou saida_existente_id na comissão, atendendo às regras de valor, competência, data e favorecido/pagador. Use gerar_entrada/gerar_saida=false para o lado vinculado. Um lote de status apenas atualiza vínculos existentes e não substitui a etapa de geração.

Erros, retornos e integração

Leia o status HTTP e o objeto error. Não trate uma resposta 204 como JSON.

HTTP	Significado	Como agir
400	Requisição inválida	Corrija JSON, ID, paginação, campo desconhecido, query desconhecida ou repetida.
401	Token ausente ou inválido	Confira Authorization: Bearer e o token configurado; a resposta informa WWW-Authenticate: Bearer.
404	Não encontrado	Confira a rota e os IDs do registro e dos cadastros relacionados.
405	Método não permitido	Use os métodos do catálogo. O cabeçalho Allow informa os aceitos; comissões não oferecem DELETE.
409	Conflito	Pode ser chave única, vínculo protegido, movimento ligado a comissão ou atualização concorrente. Consulte o estado atual antes de repetir uma gravação.
413	Corpo muito grande	Reduza o objeto JSON para no máximo 64 KB; lotes de comissões também têm limite de 100 IDs.
415	Tipo de conteúdo inválido	Envie Content-Type: application/json em POST e PUT.
422	Validação	Corrija campos obrigatórios, dinheiro, datas, status, flags e regras de negócio descritas na mensagem.
500	Erro interno	Registre horário, rota e status para suporte; o servidor não expõe detalhes sensíveis na resposta.
503	Indisponível	Confirme API_TOKEN no servidor, reinicialização e conexão ao banco.

```
{"error":{"code":"validation_error","message":"Informe uma data válida."}}
```

Os códigos de erro são strings; exemplos: unauthorized, not_found, method_not_allowed, unknown_field, validation_error, invalid_field, duplicate_record, related_record, commission_linked, concurrent_update, api_not_configured, database_unavailable e internal_error. A mensagem pode variar conforme o campo ou a regra que falhou.

Gravações e exclusões

Não há chave de idempotência. Em falha de rede depois de POST, consulte a chave natural/registro antes de reenviar para evitar duplicidade. Para comissões, a unicidade cliente/geradora/competência ajuda a identificar um registro já criado. Exclusões protegidas não são em cascata; consulte e resolva os vínculos antes de repetir.

Arquivos entregues

Arquivo	Uso
`docs/api-v1.pdf`	Manual de consumo em PDF.
`docs/api-v1.html`	Versão HTML imprimível deste manual.
`docs/api-v1.openapi.json`	Contrato OpenAPI 3.0.3 para clientes e ferramentas de API.

Este manual e a especificação são gerados por scripts/generate-api-docs.ts a partir do contrato compartilhado em src/lib/api/contract.ts. Os exemplos contêm somente dados fictícios, sem credenciais ou dados de clientes reais. A API cobre o núcleo JSON descrito; não há rotas de upload/importação CSV/XLSX, anexos ou auditoria nesta versão.